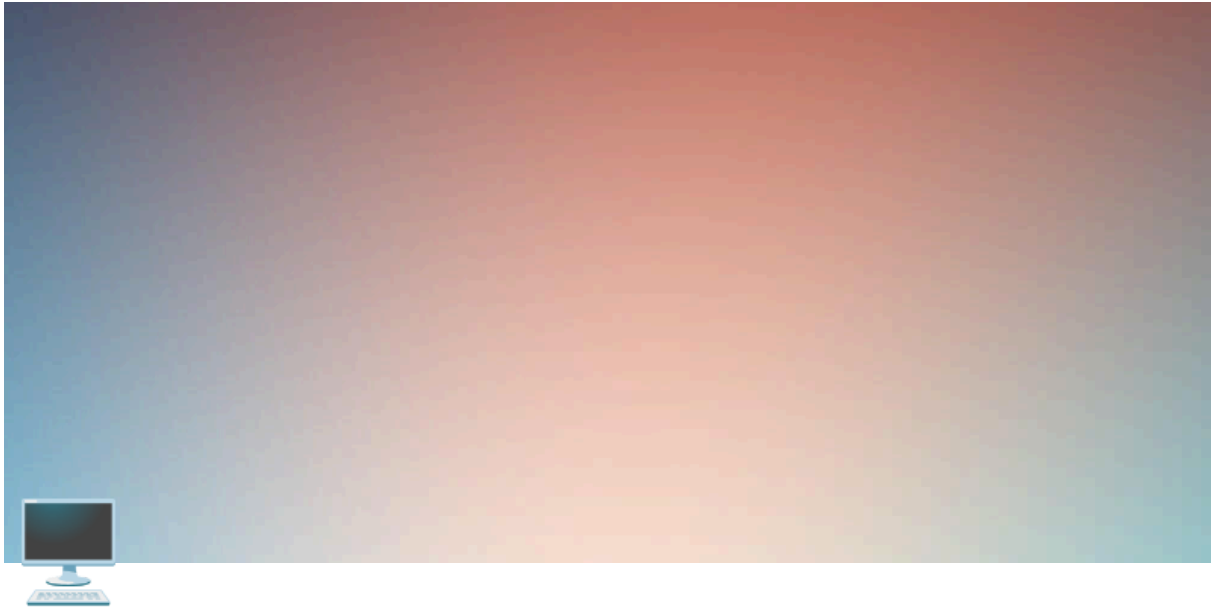




CONCEPTS RELATED TO REACT

1. What is `NPM`?

Ans: Basically, 'NPM' is basically a default package manager for Node.js, consisting of a CLI and an online registry of over 1 million open source JavaScript Packages. It enables developers to easily install, share, manage dependencies and version control code for JavaScript projects.

In simple terms, we can say It is an App Store for developers. It is a tool that allows you to download, share, manage pre-written pieces of code (called packages), so you don't need to do anything from Scratch.

FULL FORM

Originally, the full form for NPM was "Node Package Manager", but officially, the creators later said that "NPM does not actually have a full form anymore."

IMPORTANCE

- It saves time, instead of writing complex features yourself you can borrow a package from someone else in seconds.
- It also manage dependencies, if the tool you downloaded needs other tool to work then npm is smart enough to download that too.

2. What is `Parcel/Webpack`? Why do we need it?

Ans: Basically, Parcel and Webpack are tools that bundle (combine) many files of a project into optimized files that the browser can easily load.

Parcel/Webpack is like a Shipping and packing Service for the React App because the React App have different files so the browser cannot handle it efficiently.

NEED

- **PACKER:** Take all files and bundle them together into one and two clean files.
 - **TRANSLATOR:** React uses special code called JSX. Browser don't speak parcel automatically translate the JSX code to JS.
 - **LIVE PREVIEW:** Parcel run the private version of your site. Every time we hit "save". Parcel instantly updates the browser, so you can see the changes without refresh.
 - **CODE OPTIMIZATION:** Parcel automatically minifies and compresses files during production. This reduces file size and improves website performance.
 - **SUPPORT FOR MODERN WEB TECHNOLOGIES:** Parcel supports modern JavaScript, React, TypeScript, and CSS preprocessors. It handles these technologies automatically without additional setup.
-

3. What is `.parcel-cache`

Ans: `.parcel-cache` is a folder created automatically by Parcel bundler that stores cached build data to make the development and build process faster.

- It stores temporary files and processed data generated during the bundling process so Parcel does not need to rebuild everything again.
- It helps improve the build speed because Parcel can reuse previously processed files instead of compiling them from scratch.
- It contains cached information about compiled JavaScript, CSS, and project dependencies.
- It is automatically created and managed by Parcel whenever the project runs.

- It is usually added to .gitignore because it only contains temporary cache files and does not need to be uploaded to GitHub

4. What is `npx` ?

Ans: NPX is a tool that comes with NPM and is used to run Node packages without installing them globally.

- It allows you to run a package directly from the command line without installing it permanently.
- It downloads the package temporarily and runs it when needed.
- It helps avoid installing many global packages on your system.
- It is commonly used to run tools like `create-react-app` or `parcel` .

Example:

```
npx create-react-app my-app
```

This runs the package without installing it globally.

5. What is difference between `dependencies` vs `devDependencies`

Ans: `dependencies` and `devDependencies` are sections in the `package.json` file that store the packages used in a project.

Features	<code>dependencies</code>	<code>devDependencies</code>
Purpose	Packages required for the application to run.	Packages required only during development.
Usage Time	Used in production and development.	Used only during development.
Installation Command	Installed using <code>npm install package-name</code> .	Installed using <code>npm install package-name --save-dev</code> .
Example Packages	React, Express.	Parcel, Webpack.
Need in Production	Required in production to run the app properly.	Not required in production environment.

6. What is Tree Shaking?

Ans: Tree shaking is a technique used in JavaScript bundlers (like Webpack, Parcel, or Rollup) to remove unused code from the final bundle, so that the application becomes smaller and faster.

- When you import a library, sometimes you only use some functions, not all of them.
- Tree shaking removes the unused parts of that library.
- This makes the JavaScript file size smaller.
- Smaller files mean faster loading websites.

IMPORTANCE

- Reduces bundle size
 - Improves website performance
 - Removes unused code automatically
-

7. What is Hot Module Replacement?

Ans: Hot Module Replacement (HMR) is a feature in modern web development tools (like Webpack, Vite, and Parcel) that allows updating modules in the browser without reloading the whole page.

- When you change some code (like CSS, JS, or a component), the change is instantly updated in the browser.
 - The page does not fully refresh, only the changed module is replaced.
 - This helps developers see changes quickly while developing.
-

8. List down your favourite 5 superpowers of Parcel and describe any 3 of them in your own words.

Ans: 5 Favourite superpowers of Parcel are as shown below:

1. Zero Configuration
2. Hot Module Replacement (HMR)
3. Fast Bundling using Caching
4. Tree Shaking

5. Automatic Code Splitting

DESCRIPTION

1. ZERO CONFIGURATION

Parcel works without needing complicated setup or configuration files. You can start a project simply by running a command, and Parcel automatically handles bundling, transpiling, and optimization.

2. HOT MODULE REPLACEMENT

Parcel automatically updates the changed code in the browser without refreshing the entire page. This helps developers see changes instantly while developing and saves time.

3. TREE SHAKING

Tree shaking removes unused code from the final bundle. This makes the application smaller and faster because only the code that is actually used in the project is included.

9. What is `.gitignore`? What should we add and not add into it?

Ans: `.gitignore` is a special file in Git that tells Git which files or folders should be ignored and not tracked in the repository. This means Git will not upload or store those files on GitHub. If a file is listed here, Git will act as if that file does not exist for version control.

- To avoid uploading unnecessary files
- To protect sensitive data like passwords or API keys
- To keep the repository clean and lightweight

WHAT WE CAN ADD

Add files that should not be shared in the repository.

- `node_modules` → Contains installed packages (can be installed again using `npm install`)
- `.env` → Contains secret keys and passwords
- `.parcel-cache/` → Cache files created by Parcel
- `.dist/` → Generated build files
- `/log` → Log files

WHAT WE CANNOT ADD

Do not ignore files that are required to run the project.

- package.json
- package-lock.json
- Source code files (`.js` , `.jsx` , `.html` , `.css`)
- Configuration files needed by the project
- README.md

10. What is the difference between `package.json` and `package-lock.json`

Ans: `Package.json` is like the "ID-Card" of your project. It is a Single file that tells npm and other developers everything they need to now about your project. Whereas `package-lock.json` records the precise version and download location for every installed package and its sub-dependencies.

The difference between package.json and package-lock.json is as shown below

Features	<code>package.json</code>	<code>package-lock.json</code>
Purpose	Contains project information and dependency requirements.	Locks the exact versions of installed dependencies.
Created by	Created manually using <code>npm init</code> or by the developer.	Automatically generated by npm when dependencies are installed.
Contains	Project name, version, scripts, dependencies, devDependencies, etc.	Exact versions of every dependency and sub-dependency.
Editing	Developers can edit it manually.	Should not usually be edited manually.

11. Why should I not modify `package-lock.json`?

Ans: The Reasons, why we should not modify `package-lock.jsos` is as follows:

- **Automatically generated file** – It is created and updated automatically by **npm** when you install or update packages.
 - **Locks exact versions** – It stores the exact versions of dependencies and sub-dependencies to keep installations consistent.
 - **Manual changes can break dependencies** – Editing it incorrectly may cause package conflicts or errors during installation.
 - **Ensures same environment** – It helps every developer on the team install the same package versions.
 - **npm manages it safely** – If changes are needed, they should be done using commands like `npm install`, `npm update`, etc., not by editing the file directly.
-

12. What is `node\_modules` ? Is it a good idea to push that on git?

Ans: `node_modules` is a folder automatically created by Node.js / npm when you install packages using commands like: `npm install` . It contains all the libraries (packages) that your project depends on.

If your package.json has dependencies like react, then npm downloads those packages and stores them inside the `node_modules` folder.

IS IT A GOOD IDEA

No, it is not a good idea to push the code on git. Following shows the reasons why we cannot do so:

- `node_modules` contains thousands of files, so it makes the repository very large and heavy.
 - Anyone who clones the project can simply run `npm install`, and npm will recreate the `node_modules` folder automatically.
 - Because the folder has a huge number of files, pushing and pulling the repository becomes slower.
-

13. What is the `dist` folder?

Ans: `dist` folder stands for "distribution" folder.

- It is the folder where the final production-ready files of a project are stored.

- These files are generated automatically by build tools like Parcel, Webpack, or Vite.
 - The code inside dist is usually optimized, bundled, and minified so that the website loads faster.
 - It contains the files that are actually deployed to a server (HTML, CSS, JavaScript, images, etc.).
 - Developers normally do not edit files inside dist manually because they are generated from the source code.
-

14. What is **browserslist**

Ans: **browserslist** is a configuration that tells build tools which browsers your project should support.

- It defines the target browsers where your website or app should run properly.
 - Tools like Babel, Autoprefixer, and Parcel read this configuration.
 - Based on the specified browsers, these tools convert modern JavaScript or CSS into compatible code for those browsers.
 - This helps ensure your website works correctly on older or specific browsers.
-

15. Explain ^ - caret and ~ - tilde

Ans: Basically, both ^ (caret) and ~ (tilde) are used in package.json to control the version of dependencies that can be installed. Each package version follows Semantic Versioning, which has three numbers: major, minor, and patch (for example: 1.2.3). ^ allows updates to minor and patch versions, but it will not update the major version. ~ allows updates only to the patch version, but keeps the major and minor versions fixed.

So, caret (^) allows slightly bigger updates, while tilde (~) allows smaller and safer updates.

Features	Tilda (~)	caret (^)
Update allowed	Patch only	Minor and Patch both
Strictness	Very strict	Flexible

Use Case	Critical System	General development